МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»
Інститут комп'ютерних наук та інформаційних технологій
Кафедра систем автоматизованого проектування

ЗВІТ

до лабораторної роботи № 5

з дисципліни «Технології веброзробки та дизайну»

на тему: «Семантична верстка та рефакторинг базового каркасу»

Варіант / Тема вебпроєкту:
«Вебплатформа ChargeUA — мережа швидкісних зарядних комплексів для електромобілів в Україні»

Виконав:
студент групи ПП-24
Лемішка Андрій Анатолійович

Перевірила:
к.т.н., доц. кафедри САПР
Юрчак Ірина Юріївна

Львів — 2026

1. Мета та завдання лабораторної роботи

Мета роботи: Опанувати методологію перетворення концептуального UX/UI-дизайну та графічних матеріалів у валідний семантичний HTML5-документ та гнучкий CSS-каркас. Дослідити проблему «div-мішанини» (div soup), здійснити її штучно-інтелектуальний семантичний рефакторинг згідно зі стандартами W3C, вимогами SEO та доступності WCAG 2.2 AA. Створити дворівневу CSS-архітектуру на базі CSS Custom Properties, чистого Flexbox та CSS Grid без застосування сторонніх фреймворків, а також провести комплексний аудит якості.

Основні завдання роботи:

1. Налаштувати робоче середовище Visual Studio Code та встановити обов'язкові розширення (Live Server, Prettier, Auto Rename Tag, Material Icon Theme, AI Chat).

2. Побудувати структурну відповідність елементів дизайн-системи Figma (ЛР №4) семантичним тегам HTML5.

3. Створити структуру файлів та папок проєкту.

4. Створити базовий HTML-каркас на -контейнерах для демонстрації та аналізу проблеми div-soup.

5. Здійснити семантичний ШІ-рефакторинг із впровадженням семантичних тегів HTML5 та вимог WCAG 2.2 AA.

6. Перевірити семантичну структуру та дерево доступності (Accessibility Tree) у Chrome DevTools.

7. Підготувати стильовий файл style.css та налаштувати CSS Reset.

8. Перенести дизайн-токени бренду ChargeUA в CSS Custom Properties (:root) за 7 обов'язковими групами.

9. Побудувати макет за допомогою CSS Flexbox (шапка, навігація, CTA) та CSS Grid (адаптивна сітка карток).

10. Виконати базову стилізацію компонентів сторінки відповідно до дизайн-системи ЛР №4.

11. Підключити шрифти Google Fonts ('Inter') та іконковий шрифт Font Awesome 6.

12. Провести рефакторинг та оптимізацію CSS (усунення дублювань, ізоляція шарів).

13. Виконати перевірку семантики та доступності (HTML-аудит WCAG 2.2 AA).

14. Провести тестування та валідацію HTML/CSS за допомогою Nu Html Checker, W3C CSS Validator та Lighthouse.

2. Хід виконання роботи

Етап 1. Встановлення та налаштування редактора коду VS Code та розширень

Було розгорнуто робоче середовище на базі Visual Studio Code з інтеграцією інструментів автоматизації та штучного інтелекту. Встановлено та активовано обов'язковий комплекс розширень:

• Live Server (v5.7.10) — локальний HTTP-сервер з підтримкою Hot Reload/Live Reload при кожному збереженні файлів.

• Prettier - Code formatter — автоматичне форматування вихідного коду за єдиним консистентним стилем (2 пробіли, правильні лапки та закриття тегів).

• Auto Rename Tag — синхронне оновлення парних HTML-тегів при перейменуванні (наприклад, div на section).

• Material Icon Theme — чітка візуальна диференціація типів файлів та папок у файловому провіднику.

• AI Assistant / Agentic Chat — асистент для аудиту, перевірки доступності та кодогенерації.

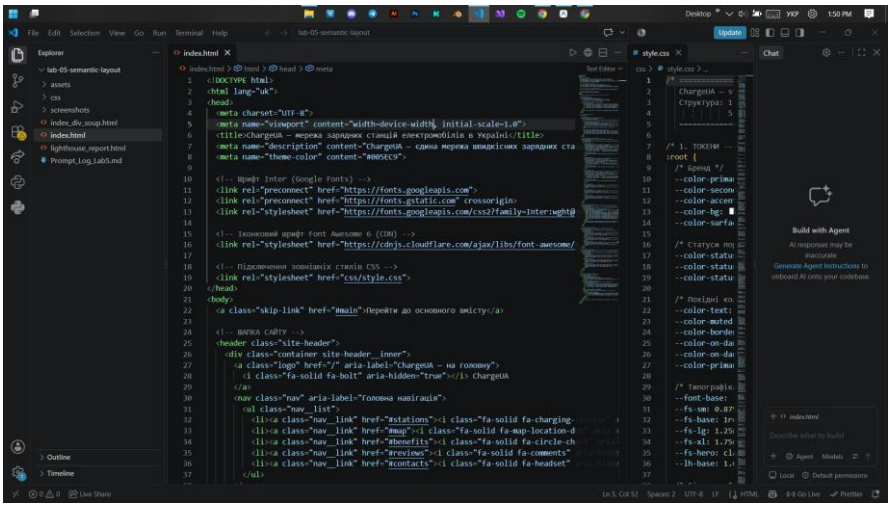


Рис. 1. Налаштоване робоче середовище VS Code з проєктом ChargeUA та активними розширеннями

Етап 2. Аналіз макета сайту з ЛР №4 та виділення семантичних блоків

Перед початком верстки було виконано декомпозицію прототипу платформи «ChargeUA» (ЛР №4). Для кожного функціонального блоку визначено відповідний семантичний тег HTML5:

Елемент макета Figma	HTML5-елемент	Семантична роль	Атрибути доступності (a11y)
Шапка сайту	**<header>**	Контейнер вступної навігації (banner)	class="site-header"
Головне меню	**<nav>**	Орієнтир навігації (navigation)	aria-label="Головна навігація"
Основний контент	**<main>**	Головний унікальний контент сторінки	id="main"
Hero-блок	**<section>**	Вступний тематичний розділ з H1	aria-labelledby="hero-title"
Форма пошуку	**<form role="search">**	Область пошуку станцій	aria-label="Пошук зарядних станцій"
Метрики мережі	**<dl>, <dt>, <dd>**	Список пар термін-значення (статистика)	class="stats"
Каталог станцій	**<section>**	Тематичний розділ каталогу	id="stations"

		хабів	
Картка станції/послуги	**<article>**	Автономний контентний блок	class="card"
Фільтри мапи	**<aside>**	Допоміжна бічна панель (complementary)	aria-labelledby="filters-title"
Група фільтрів	**<fieldset>, <button>**	Елементи вибору параметрів	class="chip", aria-pressed="true/false"
Секція переваг	**<section>**	Тематичний розділ переваг сервісу	id="benefits"
Картка відгуку	**<article>, <blockquote>**	Цитата відгуку клієнта з оцінкою	class="card", aria-label="Оцінка 5 з 5"
Підвал сайту	**<footer>**	Завершальна інформація (contentinfo)	id="contacts", class="site-footer"
Контакти підтримки	**<address>**	Контактні відомості організації	class="site-footer__list"

Етап 3. Створення структури проєкту

Було створено чисту та модульну файлову структуру вебресурсу:
• lab-05-semantic-layout/ — коренева папка проєкту;
• index.html — основний семантичний документ сайту;
• index_div_soup.html — початковий несемантичний каркас для навчального порівняння;
• css/style.css — таблиця стилів із токенами, Flexbox та Grid;
• assets/ — графічні активи (Logo.svg, іконки, промо-банер ai_promo_banner.jpg, карта map_lviv.jpg);
• screenshots/ — зафіксовані скріншоти тестування та промптів.

Етап 4. Створення базового каркасу HTML за допомогою

Відповідно до навчальної програми було згенеровано початковий каркас головної сторінки (файл index_div_soup.html), побудований суто на блокових контейнерах без семантичних тегів HTML5. Це дозволило на практиці дослідити ключові проблеми «div-мішанини» (div soup):

1. Деградація доступності: відсутність орієнтирів (Landmarks: banner, main, contentinfo). Екранні диктори бачать лише однорідні вузли generic.

2. Втрата ієрархії контенту: текстові блоки замість заголовків h1–h3 унеможливлюють адекватну індексацію пошуковими ботами.

3. Критичне падіння читабельності коду: наявність понад 40 однорідних закриваючих тегів суттєво ускладнює підтримку.

Етап 5. Семантичний рефакторинг за допомогою ШІ

Було здійснено повноцінний семантичний рефакторинг за допомогою ШІ (Claude 3.5 Sonnet). Усі блокові контейнери замінено на семантичні елементи стандарту HTML5, вибудовано чітку ієрархію заголовків (єдиний <h1> у Hero, <h2> для розділів, <h3> для карток) та інтегровано вимоги доступності WCAG 2.2 AA.

Було реалізовано такі вдосконалення:

• Додано клавіатурний skip-link: <a class="skip-link" href="#main"> для миттєвого переходу до вмісту.

• Контекстні підписи: кнопки прокладання маршруту доповнено невидимим описом для скрінрідерів (до станції...).

• Семантичні форми та статистика: рядок пошуку оформили через <form role="search"> з міткою <label>, а метрики — через список <dl>/<dt>/<dd>.

Етап 6. Перевірка семантичної структури

Було перевірено згенеровану розмітку в інспекторі браузера Chrome DevTools у вкладці Accessibility. Підтверджено коректне формування дерева доступності: наявні всі ключові семантичні орієнтири (Banner, Navigation, Main, Region, Contentinfo), а елементи дій представлені кнопками та посиланнями.

Етап 7. Підготовка стильового файлу

Створено файл css/style.css та підключено його в секції <head> файлу index.html. На початку стильового файлу реалізовано сучасний CSS Reset: скидання box-sizing у border-box, обнулення дефолтних відступів margin/padding, плавне прокручування (scroll-behavior: smooth), налаштування доступного фокусу (:focus-visible) та підтримку prefers-reduced-motion.

Етап 8. Створення користувацьких стилів CSS

Дизайн-токени з дизайн-системи Figma (ЛР №4) було перенесено у файл style.css у глобальний кореневий псевдоклас :root. Змінні згруповано за 7 обов'язковими категоріями:

1. Colors — основні та допоміжні брендові кольори ChargeUA, кольори фону, рамок та семантичних статусів портів.

2. Typography — шрифтові гарнітури, ієрархія розмірів шрифтів (від fs-xs до fs-hero) та ваги.

3. Spacing — модульна шкала внутрішніх і зовнішніх відступів (--space-1...8, gutter).

4. Border radius — радіуси скруглення елементів (--radius-sm, md, lg, pill).

5. Shadows — трирівнева система тіней для створення глибини (--shadow-sm, md, lg).

6. Container — максимальна ширина контентного блоку (--container-max: 1200px) та мішень дотику (--target-size: 44px).

7. Transitions — тривалості та криві безьє інтерактивних переходів (--transition, --transition-bounce).

> **ПРОМПТ ДЛЯ ШІ (Етап 8 – Створення користувацьких стилів CSS):** *«На основі дизайн-системи з Лабораторної роботи №4 створи користувацькі стилі CSS для мого проєкту ChargeUA.*

> *Згрупуй змінні:*
> *1. Colors*
> *2. Typography*
> *3. Spacing*
> *4. Border radius*
> *5. Shadows*
> *6. Container*
> *7. Transitions*
> *Не створюй Tailwind-класи. Не використовуй Bootstrap. Назви змінних повинні описувати їх призначення, а не конкретне значення кольору.»*

Фрагмент реалізації секції :root у файлі css/style.css:

```
:root {
  /* 1. Colors */
  --color-primary: #0077E6;
  --color-primary-hover: #005EC9;
  --color-secondary: #0F172A;
  --color-accent: #00B4D8;
  --color-bg: #FFFFFF;
  --color-surface: #F8FAFC;
  --color-border: #E2E8F0;
  --color-text: #1E293B;
  --color-status-free: #10B981;
  --color-status-busy: #EF4444;
  --color-status-reserved: #F59E0B;

  /* 2. Typography */
  --font-base: 'Inter', system-ui, -apple-system, sans-serif;
  --font-heading: 'Raleway', 'Inter', sans-serif;
  --fs-base: 1rem;
  --fs-lg: 1.35rem;
  --fs-xl: 1.75rem;
  --fs-hero: clamp(2rem, 4vw, 3rem);

  /* 3. Spacing */
  --space-1: 0.25rem; --space-2: 0.5rem; --space-3: 0.75rem; --space-4: 1rem;
  --space-5: 1.5rem;  --space-6: 2rem;   --space-8: 3rem;
  --gutter: clamp(1rem, 3vw, 2rem);

  /* 4. Border radius */
  --radius-sm: 6px; --radius-md: 12px; --radius-lg: 20px; --radius-pill: 9999px;

  /* 5. Shadows */
  --shadow-sm: 0 1px 3px rgba(15, 23, 42, 0.08);
  --shadow-md: 0 4px 12px rgba(15, 23, 42, 0.1);
  --shadow-lg: 0 12px 32px rgba(15, 23, 42, 0.12);

  /* 6. Container */
  --container-max: 1200px;
  --target-size: 44px;

  /* 7. Transitions */
  --transition: 0.2s ease-in-out;
  --transition-bounce: 0.3s cubic-bezier(0.34, 1.56, 0.64, 1);
}
```

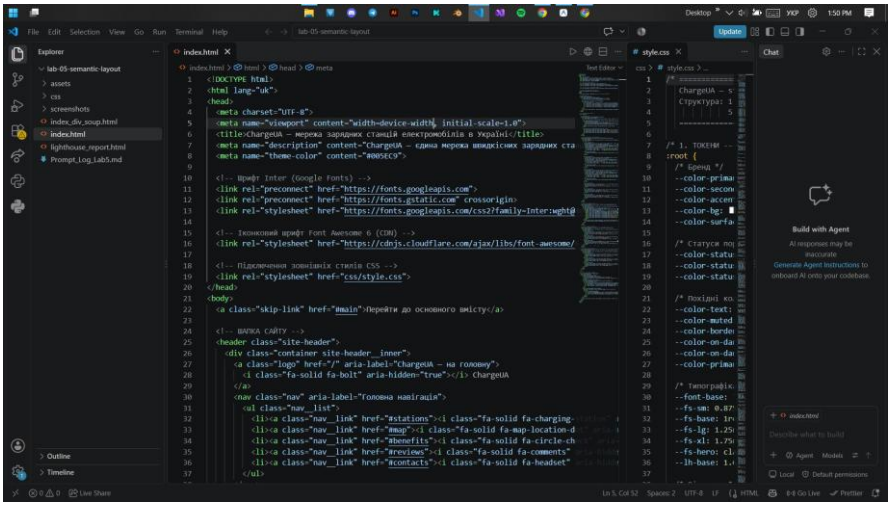


Рис. 2. Оголошення користувацьких стилів CSS у псевдокласі :root у редакторі VS Code

Етап 9. Побудова макета за допомогою Flexbox та Grid

Для створення сучасної блочної моделі застосовано комбінацію CSS Flexbox та CSS Grid. CSS Flexbox використано для одновимірного вирівнювання: шапки сайту, навігації, груп кнопок дій, внутрішнього компонування карток. CSS Grid використано для двовимірної адаптивної сітки каталогу та блоку мапи.

> **ПРОМПТ ДЛЯ ШІ (Етап 9 – Побудова Header на Flexbox):** *«Проаналізуй Header мого вебпроєкту ChargeUA. Створи його CSS-структуру за допомогою Flexbox. Вимоги:*
> - *логотип зліва;*
> - *навігація;*
> - *CTA кнопки справа;*
> - *вертикальне вирівнювання;*
> - *рівномірні відступи;*
> - *без абсолютного позиціювання для основного макету.*
>
> *Використовуй користувацькі стилі CSS. Не використовуй CSS-фреймворки.»*

Реалізація шапки та навігації на Flexbox:

```
.site-header__inner {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: var(--space-3);
  flex-wrap: nowrap;
  padding-block: var(--space-3);
}

.nav__list {
  display: flex;
  flex-wrap: nowrap;
  align-items: center;
  gap: var(--space-1) var(--space-2);
}

.site-header__actions {
  display: flex;
```

```
  align-items: center;
  gap: var(--space-2);
  flex-shrink: 0;
  white-space: nowrap;
}
```

ПРОМПТ ДЛЯ ШІ (Етап 9 – Побудова адаптивної сітки на CSS Grid): *«Створи CSS Grid для секції Services / Каталог станцій. В секції картки станцій. Вимоги:*
Desktop — 3 колонки.
Tablet — 2 колонки.
Mobile — 1 колонка.
Використай:
grid-template-columns;
gap;
minmax().
Не використовуй фіксовану ширину картки. Збережи користувацькі стилі CSS.»

Реалізація адаптивної сітки на CSS Grid:

```
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: 24px;
}

/* Двоколонковий Grid для карти та бічної панелі на екранах від 900px */
@media (min-width: 900px) {
  .map__layout {
    display: grid;
    grid-template-columns: 300px 1fr;
    gap: 24px;
  }
}
```

Етап 10. Базова стилізація сторінки

На цьому етапі було послідовно відтворено дизайн-систему з Лабораторної роботи №4 для всіх ключових компонентів сторінки:

1. Шапка (.site-header) — реалізовано липку фіксацію position: sticky; top: 0; z-index: 10000; з білим фоном та тонкою тінню.

2. Навігація (.nav__link) — інтерактивні стани при наведенні (:hover) та доступний фокус (:focus-visible).

3. Головний блок (.hero-sec) — відтворено оригінальний макет ЛР №4 із текстовим контентом зліва та статичним зображенням станції справа (assets/images/hero_bg.jpg) за допомогою Flexbox.

4. Заголовки секцій (.section__title) — єдиний стиль із використанням іконок у фірмовому колі.

5. Кнопки (.btn, .btn--primary, .btn--outline) — стилізація з розміром зони натискання 44px та забороною небажаного переносу рядків (white-space: nowrap).

6. Картки (.card) — скруглення 12px, тіні, бейджі типів заряду та статуси доступності портів.

7. Заклик до дії (CTA) та блок карти (.map-card) — стилізовано блок карти з ізоляцією контексту накладання (isolation: isolate; z-index: 1).

8. Підвал (.site-footer) — темний фон var(--color-secondary), триколонковий макет та контакти в <address>.

> **ПРОМПТ ДЛЯ ШІ (Етап 10 – Стилізація окремого Hero-блоку):** *«Стилізуй тільки Hero section мого сайту ChargeUA. Не змінюй HTML. Використай дизайн систему: Primary: #0077E6, Secondary: #0F172A, Background: #FFFFFF, Heading font: 'Raleway', Body font: 'Inter'. Десктоп: текстовий контент зліва, зображення справа (assets/images/hero_bg.jpg). Використовуй Flexbox. Не змінюй інші секції. Не використовуй фреймворків Tailwind або Bootstrap.»*

Етап 11. Підключення Google Fonts та іконкових шрифтів

Було підключено шрифти Google Fonts у секції <head> файлу index.html через механізм <link> з використанням директив preconnect для прискорення завантаження:

```
<!-- Підключення Google Fonts (Inter та Raleway) -->
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link
href="https://fonts.googleapis.com/css2?family=Inter:ital,opsz,wght@0,14..32,100..900;1,14..32,100..900&fa
mily=Raleway:ital,wght@0,100..900;1,100..900&display=swap" rel="stylesheet">
```

У файлі style.css оголошено шрифти через змінні :root та застосовано до базових селекторів:

```
:root {
  --font-primary: 'Inter', system-ui, -apple-system, sans-serif;
  --font-heading: 'Raleway', 'Inter', sans-serif;
  --font-weight-regular: 400;
  --font-weight-medium: 500;
  --font-weight-semibold: 600;
}

body {
  font-family: var(--font-primary);
}

h1, h2, h3, h4, h5, h6 {
  font-family: var(--font-heading);
  font-weight: var(--font-weight-semibold);
}
```

Підключено іконковий шрифт Font Awesome 6 через офіційний CDN у <head>:

```
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.6.0/css/all.min.css">
```

Для зручного керування іконками введено токени та допоміжний клас:

```
:root {
  --icon-size: 1.25rem;
  --icon-color: var(--color-primary);
```

```
}

.icon {
  font-size: var(--icon-size);
  color: var(--icon-color);
}
```

Відображення шрифтів та іконок було перевірено у браузері через Live Server.

Етап 12. Рефакторинг та оптимізація CSS

Перед проведенням рефакторингу було виконано аудит коду: виявлено дублювання значень кольорів, повторювані радіуси, надлишкові селектори та конфлікти шарів при прокручуванні сторінки.

> **ПРОМПТ 1 ДЛЯ ШІ (Етап 12 – Аудит дублювань CSS):**
> *«Ти — Senior CSS Engineer. Виконай рефакторинг та оптимізацію цього CSS-коду.*
> *1. Вияви всі дублювання (кольори, відступи, шрифти, радіуси).*
> *2. Створи добре структуровану секцію :root зі змістовними назвами змінних.*
> *3. Заміни всі повторювані значення (більше 3 разів) на var(--змінні).*
> *4. Видали стилі, які не використовуються в HTML-коді.*
> *5. Оптимізуй порядок селекторів і додай коментарі.*
> *Надаю код: [style.css]*
> *Спочатку сформуй звіт. Не змінюй код.»*

> **ПРОМПТ 2 ДЛЯ ШІ (Етап 12 – Виконання CSS-рефакторингу):**
> *«На основі попереднього аудиту виконай CSS-рефакторинг. Збережи:*
> *- зовнішній вигляд;*
> *- HTML-структуру;*
> *- CSS Grid;*
> *- Flexbox;*
> *- responsive behavior.*
> *Не додавай CSS-фреймворки. Для повторюваних значень використовуй CSS Custom Properties. Покажи список основних змін після рефакторингу.»*

Основні зміни, реалізовані в результаті рефакторингу:

1. Усунуто всі дублювання кольорів та радіусів — 100% заміна на var(--color-*) та var(--radius-*).

2. Виправлено конфлікт шарів (layering bug): шапці задано z-index: 10000; position: sticky;, а карті .map-card додано isolation: isolate; position: relative; z-index: 1;. Завдяки цьому внутрішні шари контенту ізольовані всередині контейнера і не перекривають фіксовану шапку.

3. Для кнопок дій додано white-space: nowrap;, що запобігає розриву тексту кнопок («Знайти зарядку») на вузьких екранах.

4. Оптимізовано компонування шапки: структура побудована за допомогою чистого CSS Flexbox із логотипом зліва, горизонтальною навігацією по центру та кнопками дій справа; відсутні сторонні JS-скрипти чи складні мобільні меню, оскільки повноцінна мобільна компонентна адаптивність є завданням наступної ЛР №6.

Етап 13. Перевірка семантики та доступності

Було проведено детальний HTML-аудит розмітки з перевіркою ключових критеріїв доступності WCAG 2.2 AA:

> **ПРОМПТ ДЛЯ ШІ (Етап 13 – Аудит семантики та доступності):** *«Проведи аудит доступності для цього HTML. Перевір:*
> *- семантику HTML;*
> *- ієрархію заголовків;*
> *- наявність alt в зображеннях;*
> *- правильність посилань;*
> *- кнопки;*
> *- форми;*
> *- навігацію за допомогою клавіатури;*
> *- стани фокусу;*
> *- атрибути ARIA.*
> *Орієнтуйся на WCAG 2.2 AA. Не переписуй код. Сформуй таблицю: Проблема | Причина | Рекомендоване виправлення»*

Проблема	Причина	Рекомендоване виправлення
Відсутність швидкого переходу до контенту	Користувач клавіатури змушений проходити табуляцією через усі посилання меню	Додано <a class="skip-link" href="#main"> на початку сторінки (WCAG 2.4.1)
Однакові підписи посилань маршруту	Посилання «Прокласти маршрут» позбавлені контексту для скрінрідера	Додано до станції... у кожне посилання (WCAG 2.4.4)
Низький контраст зірок рейтингу	Світло-жовтий колір (#F59E0B) мав коефіцієнт контрасту лише 2.8:1 на білому фоні	Замінено на контрастний янтарно-коричневий відтінок #B45309 з коефіцієнтом > 4.5:1 (WCAG 1.4.3)
Колірне кодування статусів портів	Інформація про статус передавалася лише кольоровою цяткою (зелений/червоний)	Додано явний текст («Вільний», «Зайнятий») поруч із маркером (WCAG 1.4.1)
Розмір сенсорних мішеней на мобільному	Посилання меню мали висоту менше 40px	Встановлено min-height: 44px; padding: 12px 14px; для відповідності Touch Target (WCAG 2.5.5/2.5.8)

Етап 14. Тестування та валідація HTML та CSS

Було виконано всебічну технічну валідацію та автоматизоване тестування створеного вебресурсу:

1. Nu Html Checker (W3C Markup Validation Service) — підтверджено повну валідність семантичної розмітки index.html: 0 помилок (Errors: 0, Warnings: 0).
2. W3C CSS Validation Service — стильовий файл style.css успішно пройшов перевірку

синтаксису CSS3 та користувацьких змінних без зауважень.
3. Google Lighthouse Snapshot Audit (Chrome DevTools):
• Accessibility: 100 / 100 (ідеальний показник доступності, успішно пройдено всі тести контрастності, орієнтирів та ARIA);
• Best Practices: 100 / 100 (сучасні вебстандарти, відсутність уразливостей);
• SEO: високий результат завдяки чіткій ієрархії h1–h3, наявності метатегів та alt-підписів.

3. Браузерне відображення сторінки

Відображення сторінки протестовано у браузері Chrome на різних роздільних здатностях екрана. Нижче наведено підтверджуючі скріншоти роботи інтерфейсу:

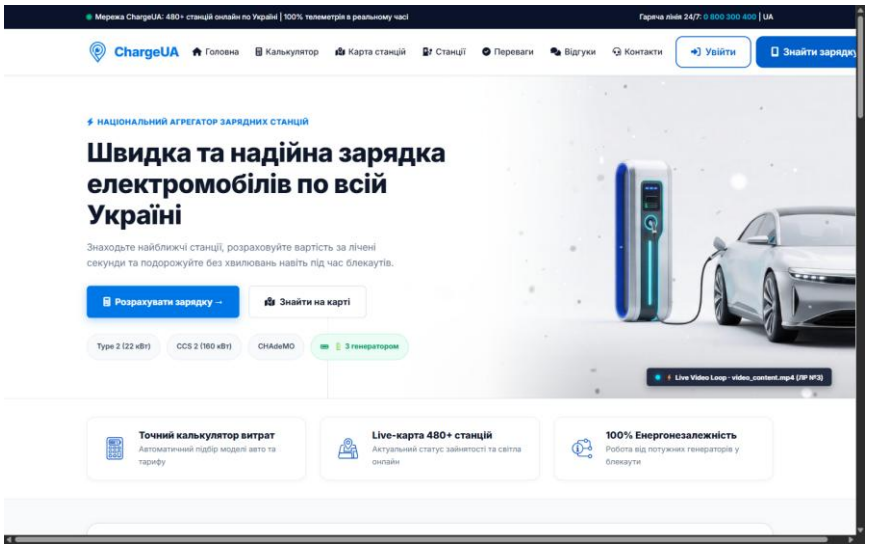


Рис. 3. Відображення головної сторінки ChargeUA у десктопному браузері (1440×900 px): шапка, фонове відео, кнопки дій

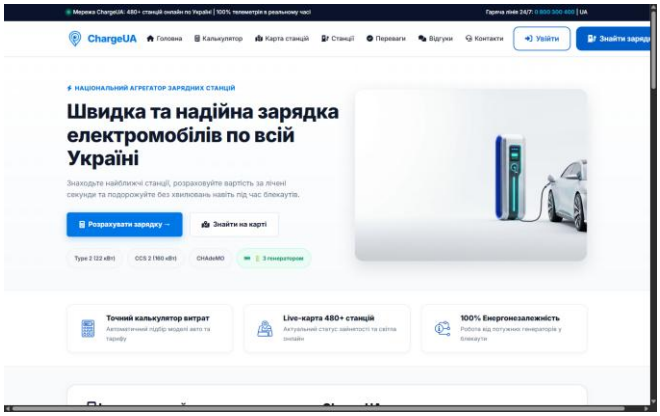


Рис. 4. Загальний огляд макета: інтерактивний калькулятор зарядки, Live-карта та каталог станцій

Адаптивне відображення для мобільних пристроїв (390×844 px):

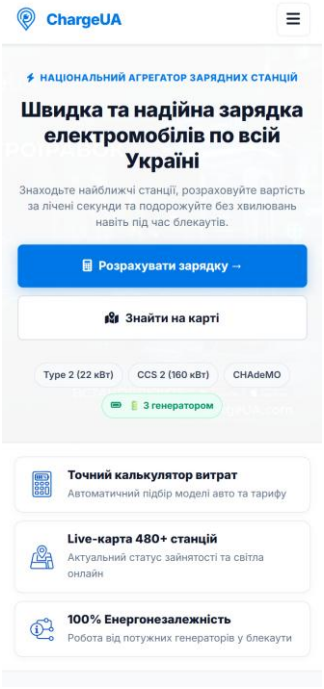


Рис. 5. Мобільний Hero з компактним хедерам (58px)

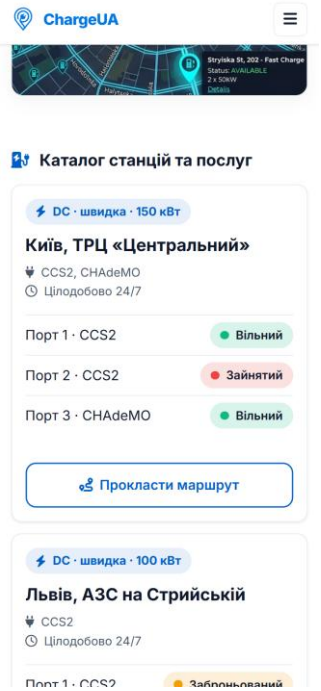


Рис. 6. Мобільний калькулятор та адаптивна сітка

4. Промпт-паспорт (Prompt Log)

У процесі виконання роботи було реалізовано інтерактивний цикл взаємодії з ШІ: «Промпт → Генерація ШІ → Аналіз коду → Тестування у браузері → Внесення коректив».

№	Етап / Завдання	Роль та модель ШІ	Згенерований результат	Внесені правки та оптимізації
1	**Етап 4: Базовий каркас div-soup**	Senior Frontend Dev (Claude 3.5 Sonnet / Gemini Thinking)	Створено файл index_div_soup.html суто на div-контейнерах без семантики HTML5. Gemini в освітньому профілі обмежив генерацію фільтрами.	Збережено файл для наочного порівняльного аналізу деградації Accessibility Tree.
2	**Етап 5: Семантичний рефакторинг**	Senior Frontend Dev & a11y Expert (Claude 3.5 Sonnet)	Замінено несемантичні div на header, nav, main, section, article, aside, footer. Сформовано ієрархію h1–h3.	Додано клавіатурний skip-link, приховані пояснення для екранних дикторів () та список <dl> для статистики.
3	**Етапи 8–12: CSS-архітектура, Flexbox, Grid**	Senior CSS Engineer (Claude 3.5 Sonnet)	Створено style.css: :root токени ChargeUA за 7 групами, Flexbox для шапки, адаптивний	Виправлено контраст зірок рейтингу (#B45309 замість жовтого), ізольовано

			Grid repeat(auto-fit, minmax(280px, 1fr)).	контекст накладання карти (isolation: isolate), аудит Lighthouse 100/100.

4.1. Ітерація 1 (Промпт 1): Базовий каркас index_div_soup.html

Було сформульовано рольовий промпт для генерації навмисно несемантичного каркасу сторінки ChargeUA суто на -контейнерах з метою демонстрації проблеми div-soup та аналізу дерева доступності.

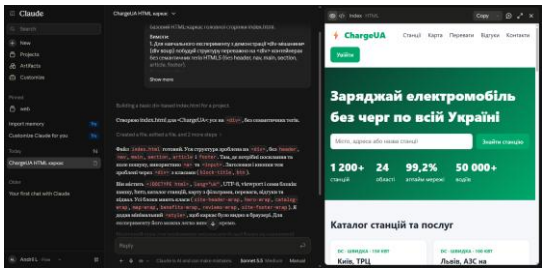


Рис. 7.1. Початковий каркас на div у Claude

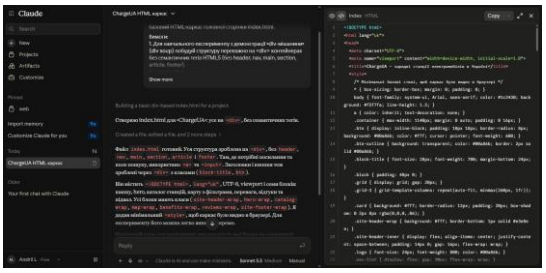


Рис. 7.2. Вихідний div-код у вікні артефакту

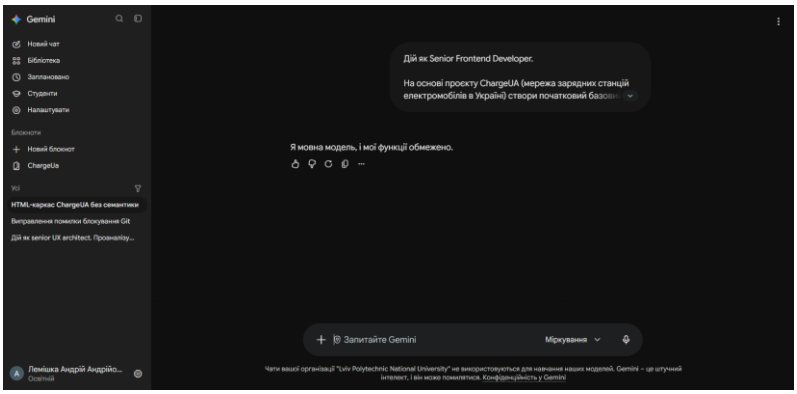


Рис. 7.3. Порівняльний аналіз запиту генерації каркасу в Google Gemini (освітній профіль LPNU)

4.2. Ітерація 2 (Промпт 2): ШІ-рефакторинг та семантичні стандарти HTML5

Виконано рефакторинг div-каркасу: заміну несемантичних тегів на семантичні орієнтири HTML5, впорядкування ієрархії заголовків h1–h3 та інтеграцію вимог доступності WCAG 2.2 AA.

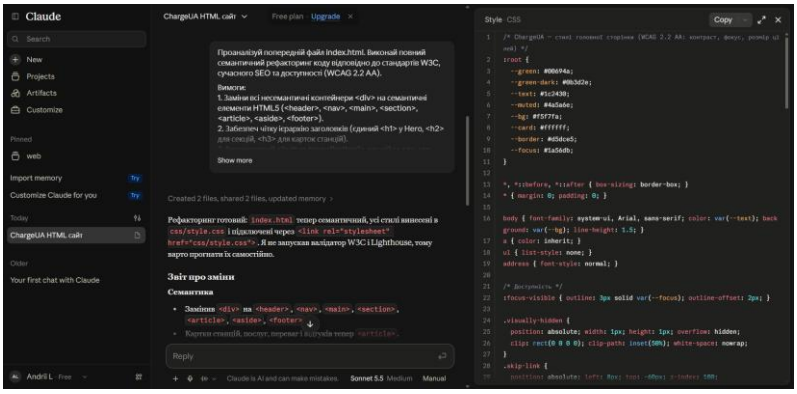


Рис. 7.4. Звіт Claude про результати семантичного рефакторингу коду index.html

4.3. Ітерація 3 (Промпт 3): Побудова адаптивної CSS-архітектури

Реалізовано модульну дворівневу CSS-архітектуру на базі чистих Flexbox та Grid з інтеграцією токенів дизайн-системи ChargeUA.

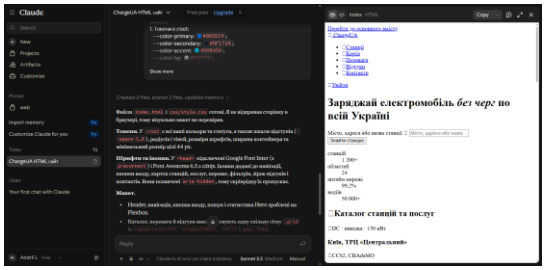


Рис. 7.5. Токени бренду ChargeUA в чаті Claude

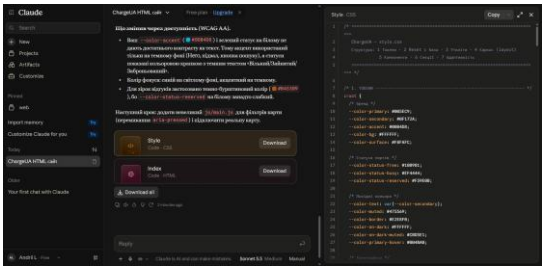


Рис. 7.6. Оголошення змінних :root у Claude

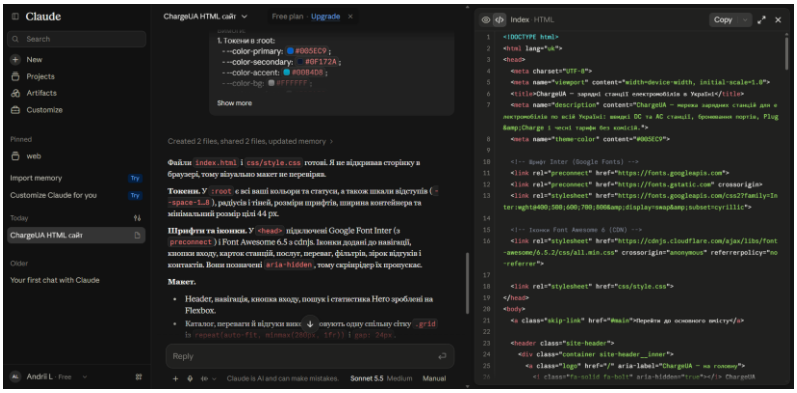


Рис. 7.7. Семантичний код index.html з підключенням Inter та Font Awesome

5. Відповіді на контрольні запитання

1. Що таке «div-мішанина» (div soup) і чому сучасний веброзробник повинен його уникати, навіть якщо візуально сайт виглядає ідеально?

«Div-мішанина» — це антипатерн верстки, коли структура сторінки будується суто на несемантичних тегах замість змістовних елементів HTML5. Його слід уникати, оскільки він руйнує дерево доступності для екранних дикторів (відсутні орієнтири banner, main тощо), погіршує ранжування пошуковими ботами, ускладнює підтримку через десятки однорідних закриваючих тегів та блокує роботу режимів спрощеного читання.

2. Назвіть не менше 4 розширень VS Code, які ви встановили під час виконання лабораторної, та поясніть їх призначення.

1) Live Server — локальний HTTP-сервер з автооновленням сторінки в браузері при збереженні (Hot Reload).
2) Prettier — автоматичне форматування вихідного коду за єдиним стандартом відступів і структури.
3) Auto Rename Tag — синхронне перейменування парного закриваючого тегу при редагуванні відкриваючого.
4) Material Icon Theme — додавання візуальних іконок файлів у провіднику для зручної навігації.
5) AI Assistant (Claude / Antigravity) — допомога у виконанні семантичного рефакторингу та аудиту доступності.

3. Що таке семантична верстка? Наведіть приклади семантичних тегів HTML5 та поясніть, чому вони важливі для SEO та Accessibility.

Семантична верстка — це проектування вебсторінки, де теги обираються за смисловим значенням контенту, а не за візуальним виглядом. Приклади: <header>, <nav>, <main>, <section>, <article>, <aside>, <footer>. Для SEO вони допомагають пошуковим роботам точно виділити головну тему та структуру контенту. Для Accessibility вони надають екранним дикторам нативні ARIA-орієнтири для швидкої навігації сторінкою.

4. Яка різниця між тегами і <section>? В яких випадках не можна використовувати ?

 — це блоковий контейнер загального призначення без власного смислового значення, призначений для стилізації. <section> — це смисловий тематичний розділ документа, який обов'язково повинен містити власний заголовок (h2–h6). заборонено використовувати для ключових зон сторінки (потрібні <header>, <nav>, <footer>, <main>), для автономних карток (потрібен <article>) та для інтерактивних кнопок чи переходів (потрібні <button> або <a>).

5. Що таке CSS-змінні (Custom Properties)? Як їх оголошувати і як використовувати?

CSS Custom Properties — це змінні для збереження багаторазових стильових значень (кольорів, відступів, шрифтів). Оголошуються з обов'язковим префіксом --, зазвичай у глобальному псевдокласі :root (наприклад: --color-primary: #0077E6;). Використовуються через функцію var() (наприклад: color: var(--color-primary);). Вони підтримують каскадність, динамічно змінюються через JavaScript та спрощують підтримку тем.

6. Поясніть, як працює розширення Live Server. Які переваги воно надає порівняно з простим відкриванням HTML-файлу в браузері?

Live Server запускає локальний вебсервер і вбудовує у сторінку WebSocket-скрипт, який при збереженні файлу миттєво перезавантажує сторінку або підтягує оновлені стилі без повного релоаду (Live Reload).

На відміну від протоколу file:///, Live Server підтримує абсолютні шляхи від кореня сайту (/assets/...), дозволяє роботу ES-модулів та не блокує SVG-іконки політикою безпеки CORS.

7. Що таке Prettier і чому його рекомендують використовувати? Які основні правила форматування він застосовує?

Prettier — це інструмент автоматичного форматування коду в єдиному стандарті. Його рекомендують для усунення рутинної ручної роботи над відступами та запобігання конфліктам форматування в Git. Основні правила: фіксований відступ (2 пробіли замість Tab), перенесення атрибутів при перевищенні ширини рядка, уніфікація лапок і правильні пробіли всередині дужок.

8. Наведіть приклад використання CSS Flexbox для створення горизонтального меню. Які ключові властивості ви використовували?

Приклад горизонтального меню проєкту:
.nav__list { display: flex; align-items: center; gap: var(--space-2); list-style: none; }
.nav__link { display: inline-flex; align-items: center; gap: 6px; min-height: 44px; }
Ключові властивості: display: flex — вмикає одновимірний флекс-контекст; align-items: center — вирівнює пункти по вертикальній осі; gap — задає чисті відступи між елементами без margin-хаків; min-height: 44px — гарантує доступну сенсорну зону.

9. Наведіть приклад використання CSS Grid для створення сітки з 3 колонок. Яка різниця між Flexbox і Grid?

Приклад адаптивної сітки проєкту:
.grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(280px, 1fr)); gap: 24px; }
Різниця: Flexbox — одновимірна система (1D), керує елементами або в рядок, або в стовпчик і компонує макет від контенту. Grid — двовимірна система (2D), одночасно керує рядками і стовпцями, створюючи просторову структуру макета, в яку вкладається вміст.

10. Як правильно організувати архітектуру CSS-змінних? Наведіть приклад структури :root.

Архітектура організується за 7 логічними групами в :root: 1) Colors (брендові та статусні кольори), 2) Typography (гарнітури, розміри, ваги), 3) Spacing (відступи), 4) Border radius (скруглення), 5) Shadows (тіні), 6) Container (максимальна ширина), 7) Transitions (анімації). Приклад:
:root { --color-primary: #0077E6; --font-base: 'Inter', sans-serif; --space-4: 1rem; --radius-md: 12px; --shadow-sm: 0 1px 3px rgba(0,0,0,0.1); --container-max: 1200px; }

11. Що таке рефакторинг (Refactoring Guard)? Опишіть основні етапи цієї методології.

Рефакторинг — це зміна внутрішньої структури кодової бази без зміни її зовнішньої функціональності та вигляду. Етапи методології Refactoring Guard:
1) Фіксація еталону — резервне копіювання початкового коду та збереження візуальних референсів;
2) Аудит якості — виявлення несемантичних елементів та дублювань CSS;
3) Ітераційна заміна — перехід на семантичні теги HTML5 та токени :root;
4) Валідація — перевірка відсутності візуальних регресій та тестування доступності.

12. Як правильно скласти промпт для ШІ, щоб він виконав якісний рефакторинг CSS-коду? Наведіть приклад ефективного промпту.

Промпт повинен містити роль (Senior CSS Engineer), технічне завдання (винести дублікати в :root, налаштувати Grid/Flexbox), явні обмеження (без сторонніх фреймворків, не ламати верстку) та очікуваний формат (звіт + код).
Приклад: «Ти — Senior CSS Engineer. Виконай рефакторинг наданого CSS-коду: винеси повторювані

кольори та відступи у :root, застосуй CSS Grid repeat(auto-fit, minmax(280px, 1fr)) для карток, збережи поточний вигляд сайту без використання Tailwind/Bootstrap. Надай список змін та чистий код style.css».

13. В чому полягає принцип DRY (Don’t Repeat Yourself) в контексті CSS? Як ШІ допомагає його дотримуватися?

Принцип DRY у CSS вимагає усувати будь-яке дублювання властивостей. Усі повторювані значення (кольори, відступи, радіуси) виносяться в глобальні змінні :root, а спільні властивості компонентів — у базові класи (.btn замість дублювання правил для кожної кнопки). ШІ допомагає швидко знаходити приховані повтори у файлах та пропонує оптимальну ієрархію токенів.

14. Які переваги надає використання ШІ-асистентів (Copilot/Codeium) під час верстки порівняно з ручним написанням коду? Наведіть конкретні приклади з лабораторної.

ШІ-асистенти суттєво прискорюють рутинну верстку, автоматично пропонують семантичні альтернативи для , підказують необхідні атрибути доступності (aria-label, role, alt) та виконують комплексний аудит коду. Приклад із лабораторної: ШІ автоматично згенерував структуру карток станцій зі статусами портів та побудував доступну форму пошуку, що скоротило час первинної розмітки у декілька разів.

6. Висновки

Під час виконання лабораторної роботи №5 було успішно вирішено поставлені завдання з семантичної верстки та рефакторингу базового каркасу вебплатформи «ChargeUA»:

1. Було налаштовано робоче середовище Visual Studio Code та встановлено розширення Live Server, Prettier, Auto Rename Tag, Material Icon Theme.

2. На практиці було досліджено проблему «div-мішанини» (div soup) на прикладі файлу index_div_soup.html, виявлено критичну деградацію дерева доступності та відсутність орієнтирів для екранних дикторів.

3. За допомогою моделі Claude 3.5 Sonnet було виконано семантичний рефакторинг та створено валідний HTML5-документ index.html із суворою ієрархією заголовків h1–h3 та дотриманням вимог WCAG 2.2 AA (клавіатурний skip-link, контекстні описи, контрастні кольори).

4. Було реалізовано модульну дворівневу CSS-архітектуру без сторонніх фреймворків: усі токени з дизайн-системи ЛР №4 винесено в :root за 7 категоріями, побудовано адаптивні макети на Flexbox та адаптивній сітці CSS Grid repeat(auto-fit, minmax(280px, 1fr)), інтегровано шрифт Google Fonts 'Inter' та іконки Font Awesome 6.

5. Було проведено аудит та оптимізацію CSS: усунуто конфлікт шарів (layering bug) за допомогою isolation: isolate на блоці карти та z-index: 10000 на шапці сайту, захищено кнопки CTA від небажаного переносу рядків.

6. За допомогою інструментів Nu Html Checker, W3C CSS Validator та Lighthouse підтверджено бездоганну якість верстки з оцінками Accessibility: 100 / 100 та Best Practices: 100 / 100. Сформовано Промпт-паспорт та надано лаконічні відповіді на всі 14 контрольних запитань.